Burst-Aware Adaptive Round Robin: Learned CPU Burst Prediction for Time Quantum Selection, Implementation and Evaluation on Linux Scheduler Traces

Kunal Srivastava
24BCE2476
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Anuj Shukla
24BCE0314
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Abstract: **Round robin (RR) scheduling depends on a single fixed time quantum, while dynamic quantum algorithms assume that the burst time of every process is known in advance. This final review implements and evaluates Burst-Aware Adaptive Round Robin (BARR), in which a lightweight learned model predicts the next CPU burst of each process from its execution history and a controller turns the prediction into a bounded per-process quantum. A labelled data set of 3.14 million CPU bursts was collected with Linux ftrace on five workload mixes, and a trace driven discrete event simulator was built with thirteen baseline and proposed policies. Gradient boosted trees reduced prediction error relative to exponential averaging by 4.9 % (mean absolute error) and 25 % (log error) on Linux bursts and by 10.6 % on synthetic bursts. The original BARR policy cut context switches by 55 % against the best fixed RR, but raised mean waiting time; we show that with exact predictions it reduces exactly to RR with the largest quantum. Using the prediction to order the queue instead (BARR-S) lowered mean waiting time by 31 % and context switches by 46 % on synthetic workloads. These are draft results from a reduced experiment.**

Keywords: *CPU scheduling, round robin, dynamic time quantum, burst prediction, XGBoost, discrete event simulation, ftrace*

I. INTRODUCTION

The short-term scheduler decides which ready process runs next and for how long. Round robin gives every process at most one time quantum before moving it to the back of the ready queue. The policy is simple and starvation free, but its behaviour depends strongly on one constant: a quantum that is too small wastes processor time on context switches, and one that is too large makes RR behave like first come first served (FCFS) [1].

Many dynamic quantum algorithms recompute the quantum from the burst times of the ready processes, for example their median [5], [6] or mean [7]. They report lower waiting time and fewer context switches, but they take the burst time of every process as an input that is known beforehand. A real operating system does not have this information at dispatch time.

Problem statement. How can a scheduler estimate the next CPU burst of a ready process from information that is really available at run time, and use that estimate to choose a time quantum that reduces waiting time, response time and context switches without starvation or noticeable prediction overhead?

The first review surveyed the literature and proposed BARR; the second review fixed the implementation plan, the evaluation parameters and success thresholds before any result was produced. This paper reports the implementation and the results. The contributions are:

- a labelled data set of 3,140,098 per-thread CPU bursts recorded with Linux ftrace on five workload mixes, with the collection and parsing scripts;
- a validated trace driven simulator with FCFS, fixed RR, SJF, SRTF, three dynamic quantum RR algorithms (in oracle and predicted mode) and BARR;
- burst predictors (exponential average, XGBoost, LSTM) with accuracy and inference latency in Python and in compiled form;
- an analysis showing why a burst-fitting quantum alone cannot beat a well tuned fixed RR, and a refined policy, BARR-S, that uses the prediction for ordering.

II. LITERATURE REVIEW AND OBJECTIVES

A. Related Work

Classical results by Liu and Layland [2] and lottery scheduling [3] show the value of policies whose behaviour can be reasoned about. Lozi et al. [4] found long-lived performance bugs in the hand-tuned Linux scheduler, which motivates learning parameters from observed behaviour [8]. Dynamic quantum RR algorithms set the quantum from the median burst of the ready queue [5], rearrange the queue around the median [6], or use the mean of remaining bursts [7]; all assume known bursts and use small hand-made process tables. Shortest job first approximates the next burst by exponential averaging [1]. History based run time prediction improves backfilling in parallel job schedulers [9], [10], and machine learning has been applied to Linux process scheduling [11]. Learning based schedulers operate at cluster level [12], [13], [14], while Chen et al. [15] placed a small neural network on the Linux load balancing path. Workload characterisation studies [16], [17] show highly heterogeneous, heavy tailed task durations. For prediction, gradient boosted trees [18] and LSTM networks [19] are the two model families considered.

B. Research Gap

Dynamic RR gains have not been checked with predicted rather than oracle burst times; prediction work targets whole batch jobs rather than millisecond CPU bursts; and no learned model has been used to choose the RR quantum with its inference cost measured.

C. Objectives

- Build a labelled data set of per-thread CPU bursts from Linux scheduler traces and synthetic workloads.
- Design lightweight predictors of the next CPU burst and compare them with exponential averaging.
- Design a quantum controller that converts the prediction into a bounded quantum without starvation.
- Evaluate the scheduler in a trace driven simulator against FCFS, SJF, SRTF, fixed RR and dynamic RR (oracle and predicted).

III. SYSTEM ARCHITECTURE

Fig. 1 shows the implemented system. The data layer records scheduler events, extracts bursts and stores them with a chronological split. The learning layer builds a feature vector from the completed bursts of a process and predicts the next burst. BARR runs inside the discrete event simulator: the quantum controller turns the prediction into a quantum, the dispatcher runs the process, and every measured burst is fed back into the history used for the next prediction. The metrics collector and statistics form the evaluation layer.

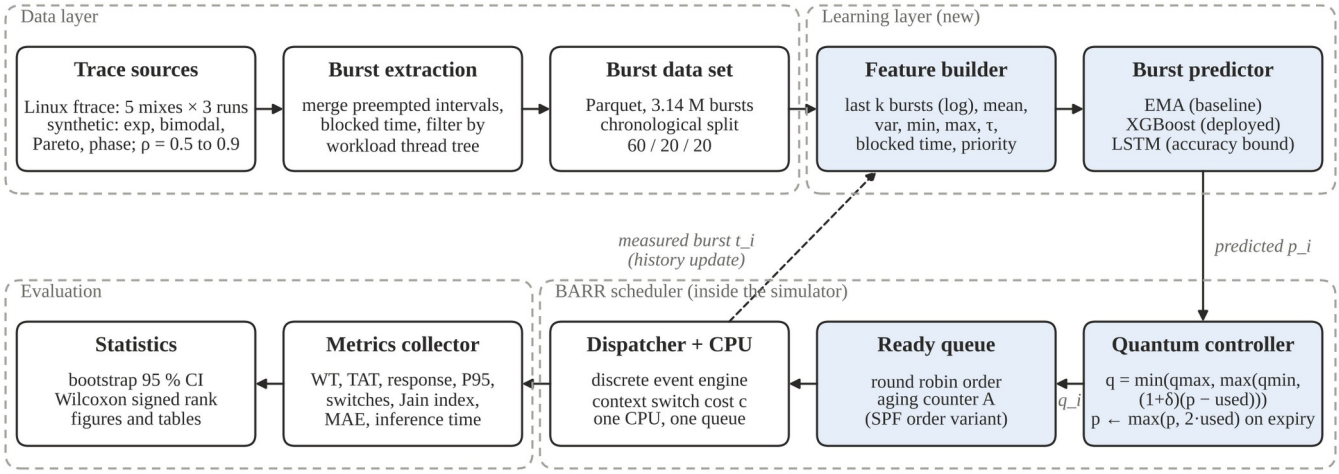


Fig. 1. Architecture of the BARR implementation. Shaded blocks are the new components; the dashed arrow is the feedback of measured bursts into the history.

IV. METHODOLOGY AND ALGORITHM

A. Burst Data Collection

Traces were recorded with ftrace on the sched_switch, sched_wakeup and sched_process_fork events while scripted workload mixes ran for 40 s, three times each: interactive (shell commands with think time), compile (parallel build of the Lua interpreter), compress (gzip, xz and ffmpeg H.264 encoding), http (Python HTTP server under ApacheBench load) and a mix of all four. The parser keeps only threads that descend from the workload root process, merges consecutive run intervals of a thread when it was switched out while still runnable (state R), and closes a burst when the thread sleeps (S, D) or exits. The time from that switch-out to the next wakeup is the blocked time before the next burst.

B. Features and Predictors

For burst i of a process the feature vector holds the last k = 8 burst lengths (log transformed, zero padded, with a fill fraction as mask), their mean, variance, minimum and maximum, the exponential average, the blocked time before the burst and the static priority. The baseline is exponential averaging,

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n \qquad (1)$$

with α chosen from {0.25, 0.5, 0.75} on validation data. The learned predictors are XGBoost regression trees on the log burst (depth 4 to 6, at most 200 trees, absolute or quantile loss chosen on validation) and a one-layer LSTM with 32 hidden units over the last 16 bursts. Accuracy is the mean absolute error,

$$MAE = (1/N)\, \Sigma\, |\, t_i - p_i\, | \qquad (2)$$

C. Quantum Controller

The quantum given to process i is

$$q_i = min(q_max,\ max(q_min,\ (1 + \delta)(p_i - u_i))) \qquad (3)$$

where p_i is the predicted burst, u_i the CPU time already used in the burst, δ = 0.25, q_min = 1 ms and q_max = 50 ms. When a quantum expires the prediction is raised to $\max(p_i, 2u_i)$, so repeated under-prediction grows the quantum geometrically up to q_max.

D. BARR Algorithm

- Step 1: when a burst becomes ready, build its features from the stored history and obtain p_i; processes with no history use the exponential average.
- Step 2: insert the process in the ready queue (FIFO for BARR; smallest $p_i - u_i$ first for BARR-S).
- Step 3: if a waiting process has been passed by more than A dispatches, move it to the head (aging).
- Step 4: dispatch the head process for q_i from (3); charge context switch cost c if it differs from the last process.
- Step 5: if the burst completes, record its length and append it to the history; otherwise raise p_i, re-queue the process and continue from Step 3.

E. Observation: Fitting the Quantum Alone Is Not Enough

With exact predictions (p_i equal to the true burst b_i) and δ ≥ 0, (3) gives $q_i \geq \min(b_i, q_max)$, so every burst runs for min(remaining, q_max), which is exactly what fixed RR with q = q_max does. FIFO BARR with a perfect predictor therefore produces the same schedule as RR with q = 50 ms, and an imperfect predictor can only add preemptions. This was confirmed by a property based test and by identical metrics in the simulator. It means the value of a burst prediction lies in deciding which process should run next rather than for how long. BARR-S applies this by serving

the smallest predicted remaining burst first, with q_max = 5 ms and aging threshold A = 64.

V. IMPLEMENTATION

A. Technology Stack

TABLE I. TECHNOLOGY STACK

Layer	Tool	Use
Language	Python 3.13	Simulator, pipeline, models
Tracing	ftrace (tracefs)	sched_switch, sched_wakeup, fork
Trace OS	Ubuntu 24.04, Linux 6.18, 2 vCPU	Trace collection
Load	make, gcc, gzip, xz, ffmpeg, ab	Workload mixes
Data	pandas, NumPy, PyArrow	Parsing, Parquet storage
Models	XGBoost 3.4, PyTorch 2.14, scikit-learn	Predictors, sanity models
Deployment check	Treelite / TL2cgen, ONNX Runtime	Compiled inference latency
Quality	pytest, Hypothesis	Unit and property tests
Analysis	SciPy, Matplotlib	Statistics, figures

B. Code Structure

The code base follows the planned layout: barr/traces (collect.sh, mixes.sh, parse_ftrace.py, load_linux.py), barr/workloads (synthetic generators), barr/features, barr/predictors (ema, gbrt, lstm, train), barr/sched (fcfs, rr, sjf, dyn_median, dyn_mean, barr), barr/sim (engine, events, process), barr/metrics, barr/experiments (YAML configurations and runner) and barr/analysis. Every scheduler implements the same four methods, on_ready, pick_next, on_preempt and on_burst_end, so the engine does not know which policy it runs.

C. Simulator

The simulator models one CPU and one ready queue with millisecond floating point time and three event kinds (arrival, slice end, I/O completion). All events at one instant are processed before the next dispatch, and every dispatch of a different process than the last one costs c = 0.1 ms.

D. Testing and Working Output

The test suite (Fig. 2) reproduces the FCFS, SJF, SRTF and RR worked examples of [1] exactly (average waiting times 17, 7, 6.5 and 5.67 ms), cross-checks the event driven RR against an independent loop on random inputs, checks with Hypothesis that the CPU never runs two processes at once, that total service equals the input bursts and that every process completes, verifies the burst parser on a hand-built ftrace snippet with a preemption, checks that no feature uses a later burst, and checks the equivalence in Section IV-E. Fig. 3 shows the demonstration program on a four process workload.

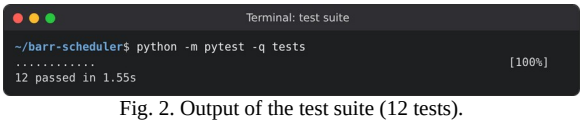


Fig. 2. Output of the test suite (12 tests).

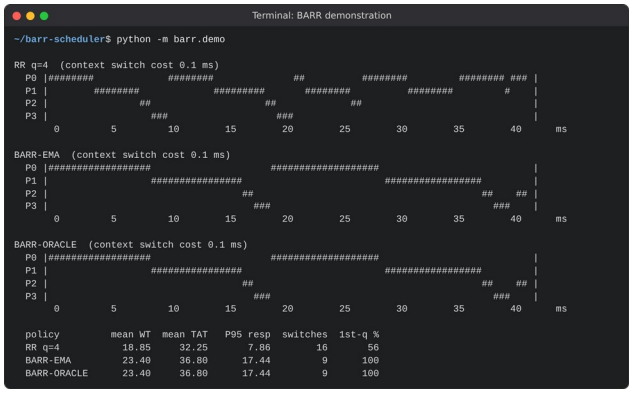


Fig. 3. Demonstration: Gantt charts of RR (q = 4 ms) and BARR on a mixed compute bound and interactive workload. BARR runs each burst to completion in one quantum (9 switches instead of 16) at the cost of longer waits for the short processes.

VI. RESULTS AND PERFORMANCE EVALUATION

A. Burst Data Set

The traces contain 3,140,098 bursts. Fig. 4 shows that bursts span six orders of magnitude: the http mix is dominated by bursts of a few microseconds, compress has a median of 37 µs, and compile has a tail of compiler bursts up to several hundred milliseconds. About 28 % of compile bursts were preempted at least once on the real machine.

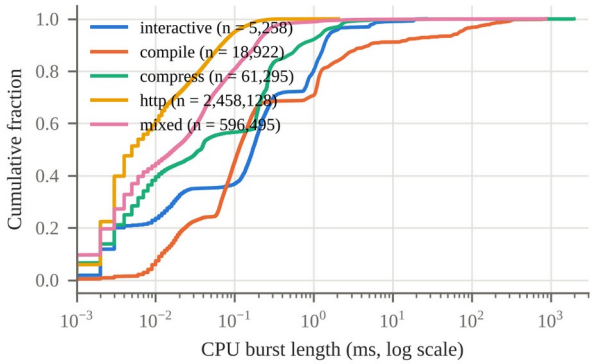


Fig. 4. Distribution of CPU burst lengths per workload mix.

B. Prediction Accuracy and Cost

TABLE II. PREDICTION ERROR ON TEST BURSTS

Predictor	Linux MAE (ms)	Linux log err.	Synth. MAE (ms)	Synth. log err.
EMA	0.186	1.34	7.64	0.63
Last value	0.195	1.38	8.08	0.66
Linear (ridge)	0.180	1.15	7.27	0.60
kNN (k=10)	0.178	1.08	7.21	0.61
GBRT (XGBoost)	0.176	1.00	6.83	0.57
LSTM	0.177	1.01	6.59	0.56

Table II and Fig. 5 compare the predictors on bursts from the last 20 % of every trace (661,689 bursts) and on 200,298 synthetic test bursts. XGBoost lowered the Linux MAE from 0.186 to 0.176 ms (4.9 %) and the log error by 25 %; the gain is largest on the http mix (23 %). On synthetic bursts the reduction is 10.6 % for XGBoost and 13.8 % for the LSTM, with the largest gain on the heavy tailed Pareto family. The planned success threshold of a 15 % MAE reduction on Linux bursts is therefore not met; MAE in milliseconds is dominated by rare long compiler bursts that no history based model anticipates.

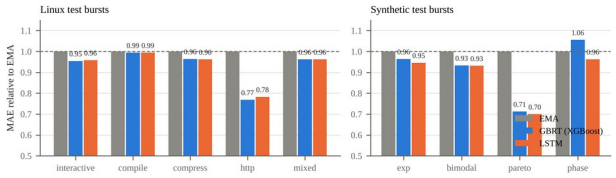


Fig. 5. Test MAE of XGBoost and LSTM relative to exponential averaging, per Linux mix and synthetic family.

Inference cost per prediction was 71 µs for XGBoost and 74 µs for the LSTM when called from Python. Compiled, the tree model took 9.5 µs per call through the Python binding and 1.5 µs per row inside the compiled code, and the LSTM took 12.7 µs in ONNX Runtime; feature construction adds about 10 µs in Python. These costs are small compared with a millisecond quantum, but not with the microsecond bursts of the http mix.

C. Scheduling Performance

TABLE III. SCHEDULING METRICS RELATIVE TO THE BEST FIXED RR (GEOMETRIC MEAN, LOWER IS BETTER)

Scheduler	Syn WT	Syn CS	Syn P95	Lin WT	Lin CS	Lin P95
FCFS	1.85	0.30	8.50	5.53	0.47	8.22
RR q=2	1.04	2.14	0.45	1.00	1.00	1.00
RR q=5	1.00	1.00	1.00	1.31	0.77	1.66
RR q=20	1.22	0.45	3.05	2.26	0.57	3.69
RR q=50	1.50	0.34	5.36	3.13	0.52	6.41
SRTF oracle	0.57	0.53	0.51	0.47	0.99	0.17
Median RR (oracle)	1.25	0.47	3.56	1.62	0.84	1.45
Median RR (pred.)	2.50	8.02	1.71	1.82	6.73	0.27
BARR-EMA	1.56	0.43	4.40	2.55	0.65	3.76
BARR-GBRT	1.57	0.45	4.21	2.55	0.65	3.76
BARR-ORACLE	1.50	0.34	5.36	3.13	0.52	6.41
BARR-S-GBRT	0.69	0.54	0.49	1.06	0.76	1.50

Table III reports a reduced draft experiment: four synthetic families at load 0.7 with three seeds, and two test-period windows from each of four Linux mixes (http omitted for time), with c = 0.1 ms. For each workload the best fixed RR is the quantum with the lowest mean waiting time (WT): q = 5 ms on all synthetic families and q = 2 ms on the Linux windows. Values are that scheduler's mean WT, context switches (CS) and 95th percentile response time divided by those of the best RR.

The original FIFO BARR reduced context switches by 55 % (synthetic) and 35 % (Linux), but increased mean WT by a factor of 1.57 and 2.55 and P95 response by about four times. BARR-ORACLE gives exactly the numbers of RR with q = 50 ms, as predicted in Section IV-E. The choice of predictor hardly matters for this policy (BARR-EMA and BARR-GBRT are within 1 %), because the quantum only needs to be larger than the burst.

BARR-S, which uses the same XGBoost predictions to order the queue, lowered mean WT by 31 %, context switches by 46 % and P95 response by 51 % relative to the best fixed RR on synthetic workloads. On the Linux windows it reduced context switches by 24 % at similar mean WT (1.06) but higher P95 response (1.50). Its maximum waiting time was 20 % (synthetic) and 42 % (Linux) above the best RR, so the aging rule needs further work. SRTF with oracle knowledge remains the lower bound on waiting time.

D. Dynamic Quantum RR with Predicted Bursts

The median quantum algorithm [5] with oracle bursts behaves well, but in predicted mode its context switches grew by a factor of 8.0 (synthetic) and 6.7 (Linux) relative to the best RR, and its mean WT rose. Once a process exceeds its predicted burst, its estimated remaining time falls to the floor and drags the median quantum down for the whole queue. The gains reported for dynamic RR therefore do not survive without oracle knowledge, which answers the research question raised in the first review.

E. Comparison with Expected Outcomes

TABLE IV. PLANNED SUCCESS THRESHOLDS

Threshold	Result
Learned MAE ≥ 15 % below EMA (Linux)	Not met: 4.9 % (log error 25 %)
BARR ≥ 20 % fewer CS than best RR, P95 within 10 %	BARR: CS met, P95 not met; BARR-S: met on synthetic, P95 not met on Linux
Max WT no worse than RR with q = q_max	BARR: met on synthetic (0.98x), met on Linux (0.87x)
Compiled inference of a few µs	Met: 1.5 µs per row (trees), 13 µs (LSTM, ONNX)

VII. DISCUSSION AND LIMITATIONS

Traces were recorded in a two-CPU virtual machine rather than on bare metal as planned, so hypervisor scheduling adds noise to burst lengths. The simulator replays two-CPU traces on one CPU, which roughly doubles the load of the compile and mixed windows. The context switch cost of 0.1 ms is large for the microsecond bursts of the http mix. Public cluster logs were not used in this draft, and the scheduling results come from a reduced matrix without confidence intervals or significance tests; the full matrix, ablations and Wilcoxon tests are prepared in the configuration files and are the next step. The preemption-fraction feature planned in the second review was dropped because it depends on the scheduler under test and would make the trained model inconsistent across policies.

VIII. CONCLUSION

This project implemented the full BARR pipeline: real Linux burst collection, a validated discrete event simulator, learned burst predictors with measured inference cost, and the proposed scheduler with all planned baselines. Learned predictors improve on exponential averaging, modestly in milliseconds and clearly in relative error, and run in a few microseconds when compiled. The main finding is that fitting the quantum to the predicted burst cannot beat a well tuned fixed quantum, because with perfect prediction it collapses to RR with the largest quantum. Using the prediction to choose which process runs next, as in BARR-S, reduced both waiting time and context switches on synthetic workloads. Future work is the full experiment with statistical tests, a stronger aging rule for BARR-S, bare metal traces and a kernel prototype of the compiled tree model.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” J. ACM, vol. 20, no. 1, pp. 46–61, 1973.

[3] C. A. Waldspurger and W. E. Weihl, “Lottery scheduling: Flexible proportional-share resource management,” in Proc. USENIX OSDI, 1994, pp. 1–11.
[4] J.-P. Lozi et al., “The Linux scheduler: A decade of wasted cores,” in Proc. EuroSys, 2016, Art. no. 1.
[5] R. J. Matarneh, “Self-adjustment time quantum in round robin algorithm depending on burst time of the now running processes,” Amer. J. Appl. Sci., vol. 6, no. 10, pp. 1831–1837, 2009.
[6] H. S. Behera, R. Mohanty, and D. Nayak, “A new proposed dynamic quantum with re-adjusted round robin scheduling algorithm and its performance analysis,” Int. J. Comput. Appl., vol. 5, no. 5, pp. 10–15, 2010.
[7] A. Noon, A. Kalakech, and S. Kadry, “A new round robin based scheduling algorithm for operating systems: Dynamic quantum using the mean average,” Int. J. Comput. Sci. Issues, vol. 8, no. 3, pp. 224–229, 2011.
[8] Y. Zhang and Y. Huang, “‘Learned’: Operating systems,” ACM SIGOPS Oper. Syst. Rev., vol. 53, no. 1, pp. 40–45, 2019.
[9] D. Tsafrir, Y. Etsion, and D. G. Feitelson, “Backfilling using system-generated predictions rather than user runtime estimates,” IEEE Trans. Parallel Distrib. Syst., vol. 18, no. 6, pp. 789–803, 2007.
[10] É. Gaussier, D. Glesser, V. Reis, and D. Trystram, “Improving backfilling by using machine learning to predict running times,” in Proc. SC, 2015, Art. no. 64.
[11] A. Negi and P. K. Kumar, “Applying machine learning techniques to improve Linux process scheduling,” in Proc. IEEE TENCON, 2005, pp. 1–6.
[12] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in Proc. ACM HotNets, 2016, pp. 50–56.
[13] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” in Proc. ACM SIGCOMM, 2019, pp. 270–288.
[14] C. Delimitrou and C. Kozyrakis, “Quasar: Resource-efficient and QoS-aware cluster management,” in Proc. ASPLOS, 2014, pp. 127–144.
[15] J. Chen, S. S. Banerjee, Z. T. Kalbarczyk, and R. K. Iyer, “Machine learning for load balancing in the Linux kernel,” in Proc. ACM APSys, 2020, pp. 67–74.
[16] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in Proc. ACM SoCC, 2012, Art. no. 7.
[17] D. G. Feitelson, D. Tsafrir, and D. Krakov, “Experience with using the Parallel Workloads Archive,” J. Parallel Distrib. Comput., vol. 74, no. 10, pp. 2967–2982, 2014.
[18] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785–794.
[19] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.
[20] The Linux Kernel Documentation, “ftrace: Function tracer.” [Online]. Available: https://docs.kernel.org/trace/ftrace.html